`k8s-06` tasks

1.Create a ConfigMap from a directory containing multiple files and inject the variables into a pod as environment variables.

1. Objective

To create a Kubernetes ConfigMap from a directory containing multiple files and inject the configuration values into a Pod as environment variables.

2. Environment

- Kubernetes cluster: AWS EC2
- Operating System: Ubuntu 24.04.5
- Kubernetes version: v1.37.2
- Nodes: 1 Master and 2 Workers
- Container image: `busybox:1.36`

3. Create a directory with multiple configuration files

First, create a directory named `app-config`.

```
None
mkdir app-config
cd app-config
```

Create the first configuration file:

```
None
nano app.properties
```

Add:

```
None
APP_MODE=production
APP_COLOR=blue
```

Create the second configuration file:

```
None
nano database.properties
```

Add:

```
None
DB_HOST=mysql-service
DB_PORT=3306
```

Verify the files:

```
None
ls -l
cat app.properties
cat database.properties
```

Explanation: We created two files to store application and database configuration separately.

4. Create ConfigMap from the directory

Execute:

```
None
kubectl create configmap app-directory-config --from-file=.
```

Verify:

```
None
kubectl describe configmap app-directory-config
```

Explanation:

- `create configmap` creates a ConfigMap.
- `app-directory-config` is the ConfigMap name.
- `--from-file=.` reads files from the current directory.

Important: Each file becomes one ConfigMap key. For example, `app.properties` becomes one key containing its complete contents. Its individual lines do not automatically become separate environment variables.

5. Create a directory containing individual environment variable files

To inject four separate environment variables using `envFrom`, create another directory:

```
None
cd ~
mkdir app-env
cd app-env
```

Create four files:

```
None
echo -n "production" > APP_MODE
echo -n "blue" > APP_COLOR
echo -n "mysql-service" > DB_HOST
echo -n "3306" > DB_PORT
```

Verify:

```
None
ls -l
```

6. Create ConfigMap from the environment variable directory

Execute:

```
None
kubectl create configmap app-env-config
--from-file=/home/ubuntu/app-env/
```

Verify:

```
None
kubectl describe configmap app-env-config
```

Expected ConfigMap keys and values:

Key	Value
APP_MODE	production
APP_COLOR	blue
DB_HOST	mysql-service
DB_PORT	3306

7. Create Pod YAML file

Create the YAML file:

```
None
nano app-config-pod.yaml
```

Add the following configuration:

```
None
apiVersion: v1
kind: Pod
metadata:
  name: app-config-pod
  labels:
```

```
        app: config-demo
        tier: backend
    spec:
      containers:
        - name: config-container
          image: busybox:1.36
          command: ["sh", "-c", "sleep 3600"]
          envFrom:
            - configMapRef:
                name: app-env-config
```

Explanation of important fields:

Field	Purpose
`apiVersion: v1`	Kubernetes API version
`kind: Pod`	Creates a Pod
`metadata.name`	Pod name
`image`	Container image
`command`	Keeps the container running

`envFrom`	Imports all eligible ConfigMap keys as environment variables
`configMapRef`	References the ConfigMap

The label `tier: backend` also avoids interference from an existing ReplicaSet selector in this lab environment.

8. Apply the Pod YAML file

Execute:

```
None
kubectl apply -f app-config-pod.yaml
```

Expected output:

```
None
pod/app-config-pod created
```

Check Pod status:

```
None
kubectl get pods -o wide
```

Observed result:

Pod	Ready	Status	Node
app-config-pod	1/1	Running	k8s-worker-1

9. Verify environment variables inside the Pod

Execute:

```
None
      kubectl exec app-config-pod -- printenv
```

Observed output included:

```
None
      DB_HOST=mysql-service
      DB_PORT=3306
      APP_COLOR=blue
      APP_MODE=production
```

All four values were successfully injected into the container.

You can also verify each variable separately:

```
None
      kubectl exec app-config-pod -- printenv APP_MODE
      kubectl exec app-config-pod -- printenv APP_COLOR
      kubectl exec app-config-pod -- printenv DB_HOST
      kubectl exec app-config-pod -- printenv DB_PORT
```

10. How does it work?

Directory: app-env

APP_MODE, APP_COLOR, DB_HOST, DB_PORT

ConfigMap: app-env-config

Pod: app-config-pod

`envFrom` imports ConfigMap values

Container environment variables

`APP_MODE=production`

```
APP_COLOR=blue

DB_HOST=mysql-service

DB_PORT=3306
```

11. Important concepts learned

1. ConfigMap: Stores non-sensitive application configuration.
2. `--from-file`: Creates ConfigMap keys from files.
3. `envFrom`: Imports multiple ConfigMap keys into a container as environment variables.
4. `configMapRef`: Connects a Pod to an existing ConfigMap.
5. `kubectl exec`: Executes commands inside a running container.
6. `printenv`: Displays environment variables inside the container.

12. Real-time DevOps example

Imagine an application needs database connection details:

```
None
DB_HOST=mysql-service
DB_PORT=3306
APP_MODE=production
```

Instead of hardcoding these values in the application image, a DevOps engineer can store them in a ConfigMap and inject them into the Pod.

This makes configuration easier to manage across development, testing and production environments.

Important: ConfigMaps are for non-sensitive values. Passwords and credentials should be stored in Kubernetes Secrets.

13. Final verification checklist

Created directory with multiple configuration files.

Created ConfigMap from the directory.

Verified ConfigMap contents.

Created environment variable files.

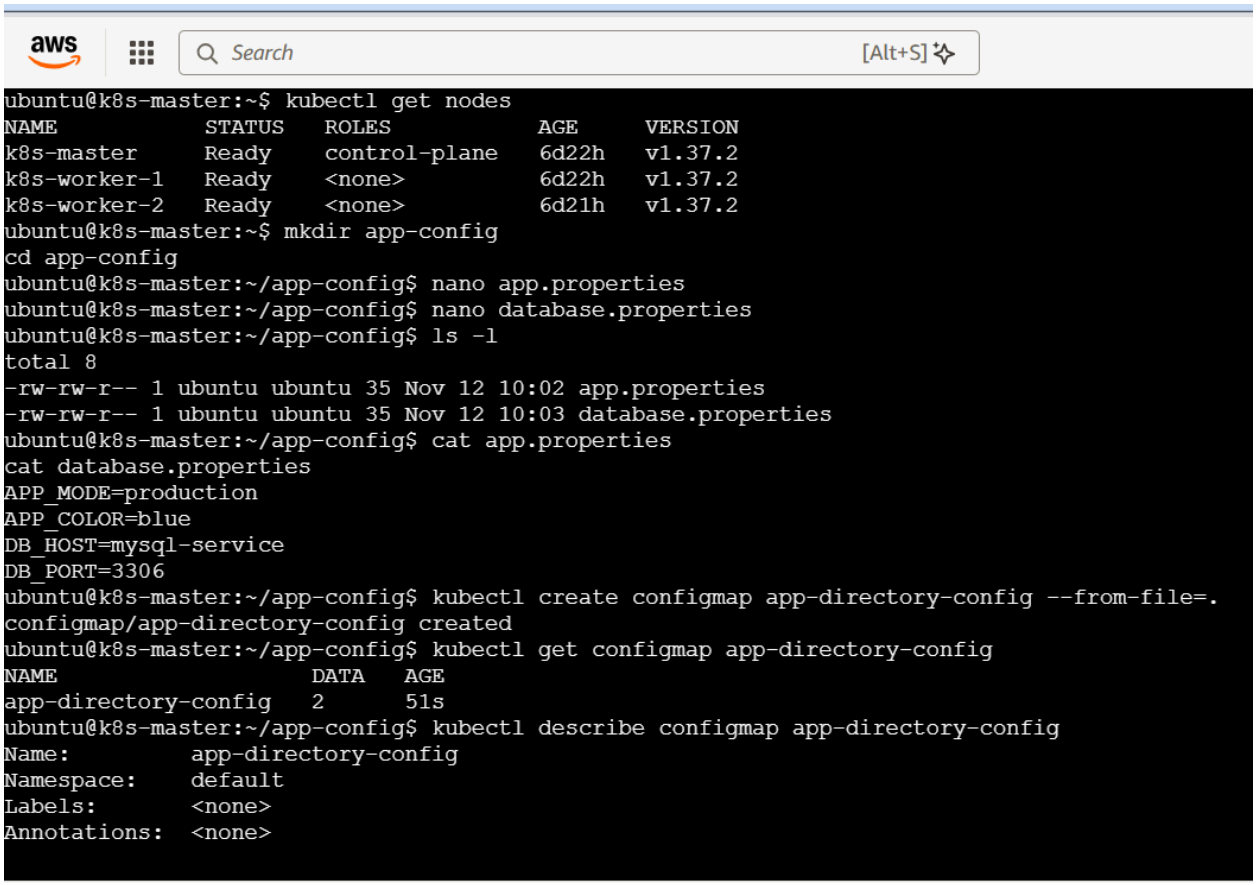
Created `app-env-config`.

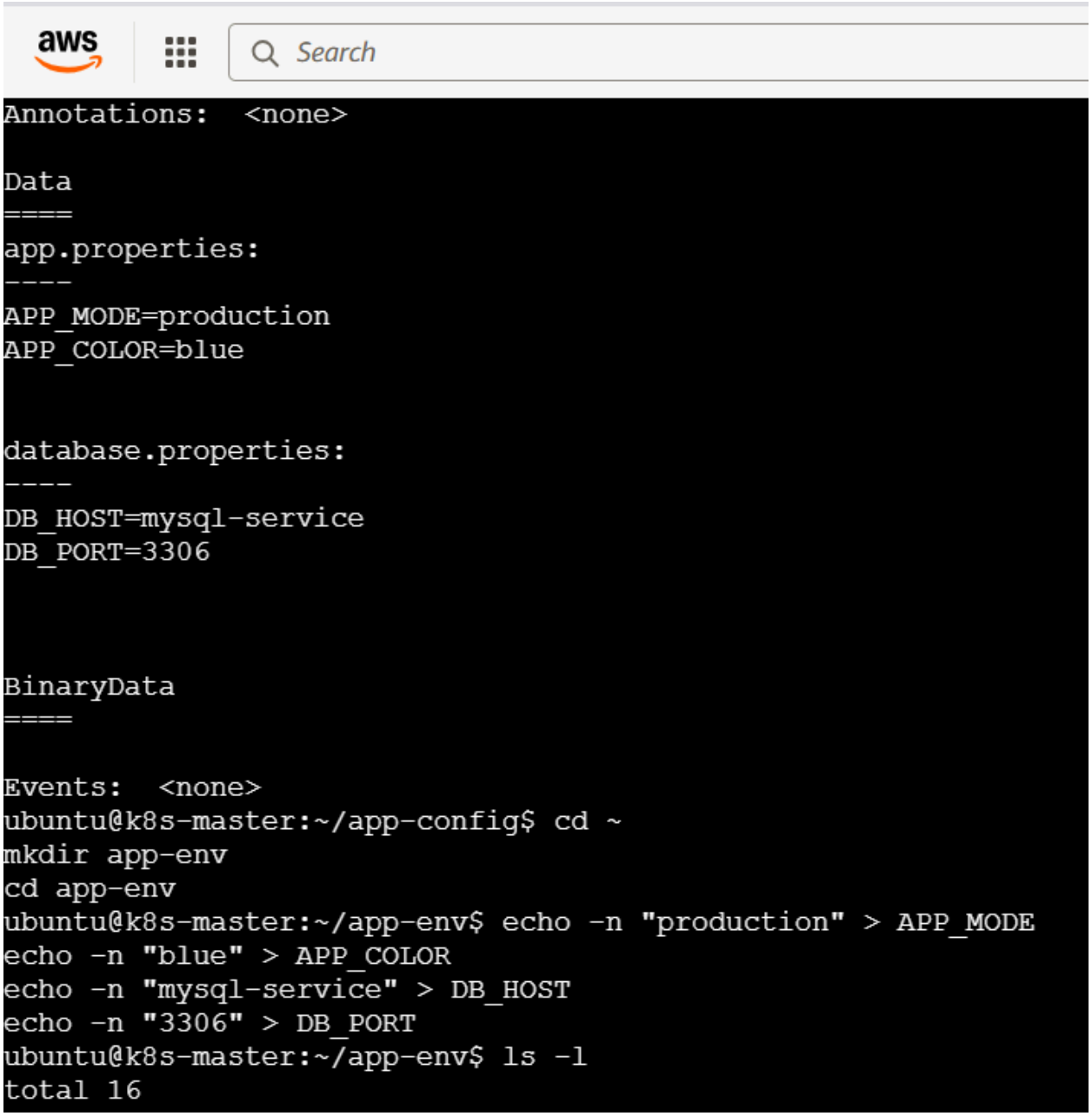
Created Pod YAML with `envFrom`.

Applied Pod successfully.

Verified Pod is Running.

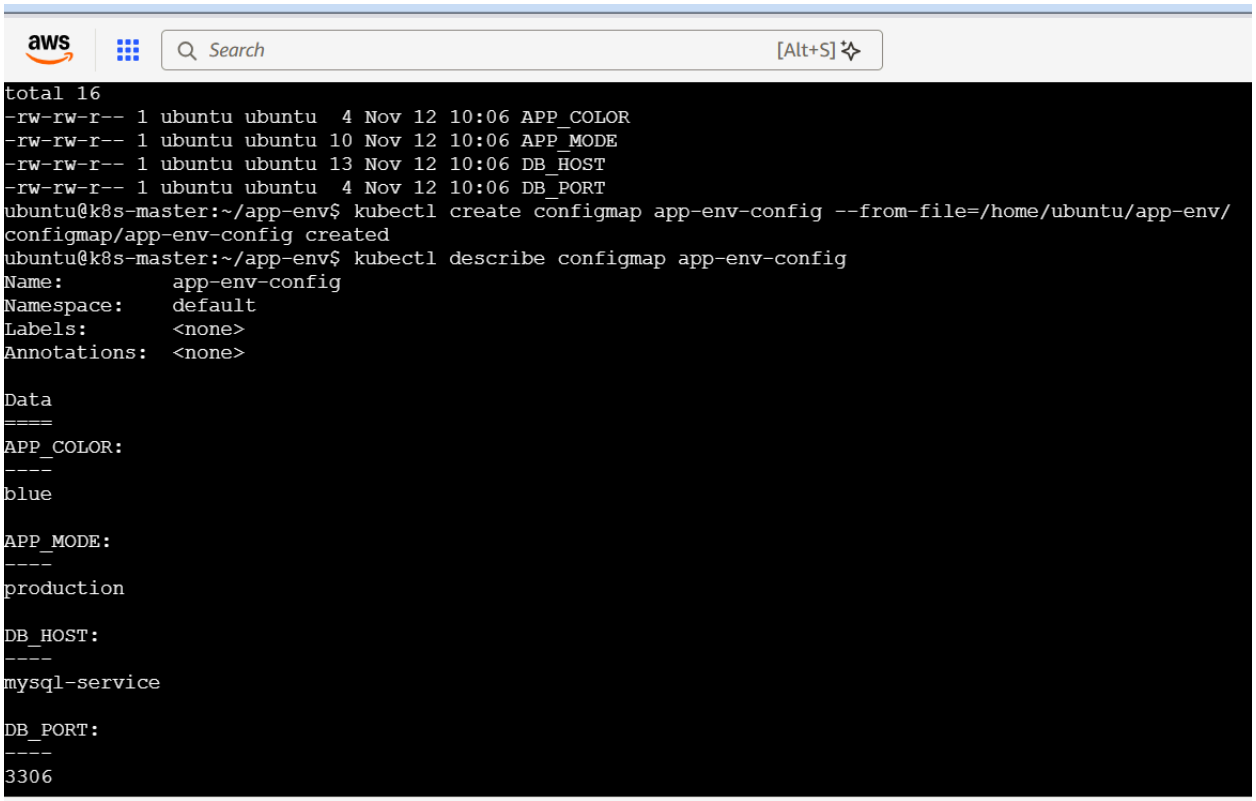
Verified all four environment variables inside the container.




aws
Search
Annotations: <none>
Data
====
app.properties:

APP_MODE=production
APP_COLOR=blue
database.properties:

DB_HOST=mysql-service
DB_PORT=3306
BinaryData
====
Events: <none>
ubuntu@k8s-master:~/app-config$ cd ~
mkdir app-env
cd app-env
ubuntu@k8s-master:~/app-env$ echo -n "production" > APP_MODE
echo -n "blue" > APP_COLOR
echo -n "mysql-service" > DB_HOST
echo -n "3306" > DB_PORT
ubuntu@k8s-master:~/app-env$ ls -l
total 16



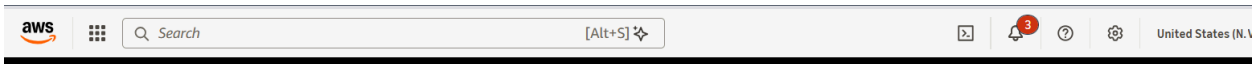

```
total 16
-rw-rw-r-- 1 ubuntu ubuntu  4 Nov 12 10:06 APP_COLOR
-rw-rw-r-- 1 ubuntu ubuntu 10 Nov 12 10:06 APP_MODE
-rw-rw-r-- 1 ubuntu ubuntu 13 Nov 12 10:06 DB_HOST
-rw-rw-r-- 1 ubuntu ubuntu  4 Nov 12 10:06 DB_PORT
ubuntu@k8s-master:~/app-env$ kubectl create configmap app-env-config --from-file=/home/ubuntu/app-env/
configmap/app-env-config created
ubuntu@k8s-master:~/app-env$ kubectl describe configmap app-env-config
Name:         app-env-config
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
APP_COLOR:
----
blue

APP_MODE:
----
production

DB_HOST:
----
mysql-service

DB_PORT:
----
3306
```




```
----
3306


BinaryData
====

Events:  <none>
ubuntu@k8s-master:~/app-env$ nano app-config-pod.yaml
ubuntu@k8s-master:~/app-env$ kubectl apply -f app-config-pod.yaml
pod/app-config-pod created
ubuntu@k8s-master:~/app-env$ kubectl get pods -o wide
NAME                                READY   STATUS      RESTARTS      AGE     IP             NODE           NOMINATED NODE   READINESS GATES
app-config-pod                      1/1     Running     0             21s     10.244.1.72    k8s-worker-1   <none>           <none>
nginx-69b9cdbbdd-ckl4s              1/1     Running     5 (31m ago)   6d22h   10.244.1.64    k8s-worker-1   <none>           <none>
nginx-69b9cdbbdd-mb55d              1/1     Running     5 (31m ago)   6d22h   10.244.2.70    k8s-worker-2   <none>           <none>
nginx-dns-pod                       1/1     Running     1 (31m ago)   23h     10.244.2.65    k8s-worker-2   <none>           <none>
nginx-history-66b76ccf76-n8ss9      1/1     Running     2 (31m ago)   47h     10.244.1.69    k8s-worker-1   <none>           <none>
nginx-history-66b76ccf76-s9nr7      1/1     Running     2 (31m ago)   47h     10.244.2.59    k8s-worker-2   <none>           <none>
nginx-pause-684c598bfc-6mcpn        1/1     Running     2 (31m ago)   46h     10.244.2.62    k8s-worker-2   <none>           <none>
nginx-pause-684c598bfc-nxlr6        1/1     Running     2 (31m ago)   46h     10.244.1.67    k8s-worker-1   <none>           <none>
nginx-pause-684c598bfc-pm7cr        1/1     Running     2 (31m ago)   46h     10.244.2.58    k8s-worker-2   <none>           <none>
nginx-rc-4gxgc                      1/1     Running     3 (31m ago)   2d22h   10.244.1.66    k8s-worker-1   <none>           <none>
nginx-rc-97jc9                      1/1     Running     3 (31m ago)   2d22h   10.244.2.60    k8s-worker-2   <none>           <none>
nginx-rc-rvqmg                      1/1     Running     3 (31m ago)   2d22h   10.244.2.64    k8s-worker-2   <none>           <none>
nginx-recreate-75844d7d6b-gwkz2     1/1     Running     2 (31m ago)   2d      10.244.1.68    k8s-worker-1   <none>           <none>
nginx-recreate-75844d7d6b-jnxcl     1/1     Running     2 (31m ago)   2d      10.244.1.62    k8s-worker-1   <none>           <none>
nginx-recreate-75844d7d6b-vdfwz     1/1     Running     2 (31m ago)   2d      10.244.2.71    k8s-worker-2   <none>           <none>
nginx-rollback-db69c556c-29j7l      1/1     Running     2 (31m ago)   47h     10.244.1.70    k8s-worker-1   <none>           <none>
nginx-rollback-db69c556c-dj8h4      1/1     Running     2 (31m ago)   47h     10.244.2.68    k8s-worker-2   <none>           <none>
```

```
nginx-surge-5fc74fd689-wd5vz      1/1     Running     2 (31m ago)   47h     10.244.1.65   k8s-worker-1   <none>           <none>
recreate-client                   0/1     Completed   0             2d      <none>        k8s-worker-2   <none>           <none>
resource-scheduler-pod            1/1     Running     2 (31m ago)   46h     10.244.2.72   k8s-worker-2   <none>           <none>
ubuntu@k8s-master:~/app-env$ kubectl exec app-config-pod -- printenv APP_MODE
production
ubuntu@k8s-master:~/app-env$ kubectl exec app-config-pod -- printenv APP_COLOR
blue
ubuntu@k8s-master:~/app-env$ kubectl exec app-config-pod -- printenv DB_HOST
mysql-service
ubuntu@k8s-master:~/app-env$ kubectl exec app-config-pod -- printenv DB_PORT
3306
ubuntu@k8s-master:~/app-env$ kubectl exec app-config-pod -- printenv
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=app-config-pod
DB_HOST=mysql-service
DB_PORT=3306
APP_COLOR=blue
APP_MODE=production
NGINX_NODEPORT_DNS_PORT=tcp://10.106.242.119:80
NGINX_NODEPORT_DNS_PORT_80_TCP_ADDR=10.106.242.119
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
NGINX_RECREATE_SVC_PORT=tcp://10.101.224.31:80
NGINX_SERVICE_SERVICE_PORT=80
NGINX_SERVICE_PORT_80_TCP=tcp://10.97.163.85:80
KUBERNETES_PORT_443_TCP_PORT=443
NGINX_LB_NODEPORT_PORT_80_TCP_PROTO=tcp
NGINX_NODEPORT_DNS_SERVICE_HOST=10.106.242.119
NGINX_RECREATE_SVC_PORT_80_TCP=tcp://10.101.224.31:80
NGINX_SERVICE_PORT_80_TCP_PROTO=tcp
NGINX_RECREATE_SVC_PORT_80_TCP_ADDR=10.101.224.31
```

2.Create a ConfigMap from a file and mount it as a volume inside a pod, ensuring the configuration data is available as files.

1. Objective

To create a ConfigMap from an existing configuration file and mount that ConfigMap inside a Kubernetes Pod as a volume. The application should be able to access the configuration data as a file inside the container.

2. Environment

- Platform: AWS EC2
- OS: Ubuntu 24.04.5
- Kubernetes: v1.37.2
- Cluster: 1 Master and 2 Worker Nodes
- Container image: `busybox:1.36`

3. Create a configuration file

We used the existing directory:

```
None
```

```
cd ~/app-config
```

The configuration file `app.properties` contains:

```
None
APP_MODE=production
APP_COLOR=blue
```

Verify the file:

```
None
ls -l
cat app.properties
```

4. Create ConfigMap from the file

Execute:

```
None
kubectl create configmap app-file-config
--from-file=app.properties
```

The ConfigMap already existed from an earlier attempt, so Kubernetes returned:

```
None
error: failed to create configmap: configmaps
"app-file-config" already exists
```

We verified the existing ConfigMap:

```
None
kubectl get configmap app-file-config
```

Output:

```
None
NAME              DATA   AGE
```

```
app-file-config   1      14m
```

Check its contents:

```
None
kubectl describe configmap app-file-config
```

Observed data:

```
None
Data
====
app.properties:
----
APP_MODE=production
APP_COLOR=blue
```

Explanation:

- `app-file-config` is the ConfigMap name.
- `DATA: 1` means one key is stored.
- `app.properties` is the ConfigMap key.
- The file contents are stored as the value.

5. Create Pod YAML file

Create the YAML file:

```
None
nano config-volume-pod.yaml
```

Add the following configuration:

None

```
apiVersion: v1
kind: Pod
metadata:
  name: config-volume-pod
  labels:
    app: config-volume-demo
    tier: backend
spec:
  containers:
    - name: config-container
      image: busybox:1.36
      command: ["sh", "-c", "sleep 3600"]
      volumeMounts:
        - name: config-volume
          mountPath: /etc/app-config
          readOnly: true
  volumes:
    - name: config-volume
      configMap:
        name: app-file-config
```

Save and exit:

- Press `Ctrl + O`
- Press Enter
- Press `Ctrl + X`

6. Understand the YAML configuration

YAML field	Explanation

`kind: Pod`	Creates a Kubernetes Pod
`containers`	Defines the container
`image: busybox:1.36`	Uses BusyBox image
`command`	Keeps the container running
`volumeMounts`	Mounts the volume inside the container
`name: config-volume`	Volume name
`mountPath`	Location inside the container
`readOnly: true`	Makes the mounted directory read-only
`volumes`	Defines the Pod volume
`configMap`	Uses ConfigMap as the volume source
`name: app-file-config`	References the existing ConfigMap

7. Apply the Pod YAML

Execute:

```
None
kubectl apply -f config-volume-pod.yaml
```

Observed output:

```
None
pod/config-volume-pod created
```

8. Verify Pod status

Execute:

```
None
kubectl get pod config-volume-pod
```

Observed output:

```
None
NAME                READY   STATUS    RESTARTS   AGE
config-volume-pod   1/1     Running   0          5m26s
```

Result: Pod is running successfully with one container ready and zero restarts.

9. Verify the mounted configuration file

Execute:

```
None
kubectl exec config-volume-pod -- ls -l /etc/app-config
```

This checks whether the configuration file exists inside the container.

Read the file:

```
None
kubectl exec config-volume-pod -- cat
/etc/app-config/app.properties
```

Observed output:

```
None
APP_MODE=production
APP_COLOR=blue
```

Successfully verified

The configuration file is available inside the container at:

```
None
/etc/app-config/app.properties
```

10. How does ConfigMap volume mounting work?

Configuration file

`app.properties`

ConfigMap

`app-file-config`

Kubernetes Pod

`volumes` + `volumeMounts`

Container

`/etc/app-config/app.properties`

11. Task 1 vs Task 2

Feature	Task 1	Task 2

ConfigMap injection	Environment variables	Volume mount
YAML configuration	`envFrom`	`volumes` and `volumeMounts`
Access command	`printenv`	`cat`
Data availability	Environment	File system
Example	`APP_MODE=production`	`app.properties`

12. Real-time DevOps scenario

Imagine an application needs a configuration file containing:

- Application mode
- Database connection details
- Logging configuration

Instead of keeping these values inside the container image, a DevOps engineer can store them in a ConfigMap and mount the ConfigMap as a file inside the container.

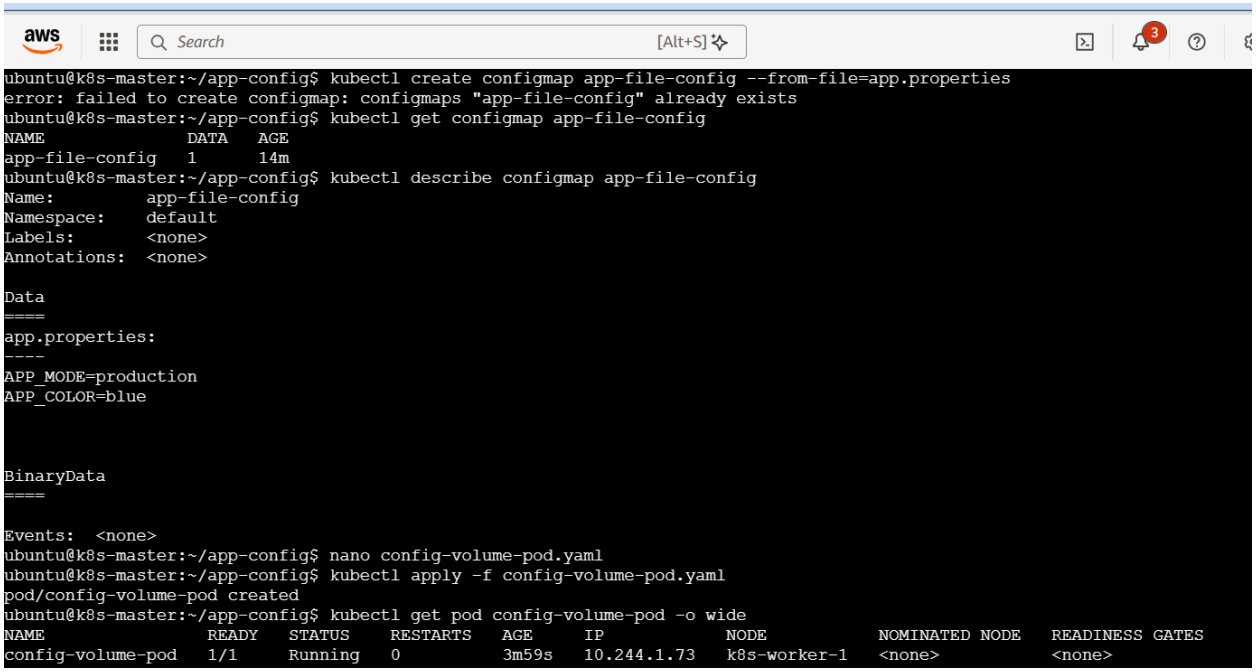
This allows configuration to be managed separately from application code.

Important: Mounted ConfigMap files can update eventually when the ConfigMap changes. Applications may need to reload the file. A `subPath` mount does not receive automatic updates.

13. Final verification checklist

- Created configuration file.
- Created ConfigMap from the file.
- Verified ConfigMap contents.
- Created Pod YAML.
- Configured volume and volumeMounts.
- Applied Pod successfully.

- Verified Pod is Running.
- Read configuration file from inside the container.





K8S-06 Task 2 successfully completed.

3.Create a Secret with sensitive information (username and password) and inject it into a pod as environment variables.

1. Objective

To create a Kubernetes Secret containing username and password credentials and inject those values into a Pod as environment variables.

2. Environment

- Platform: AWS EC2
- OS: Ubuntu 24.04.5
- Kubernetes: v1.37.2
- Cluster: 1 Master and 2 Worker Nodes
- Container image: `busybox:1.36`

3. Create Kubernetes Secret

Execute:

```
None
      kubectl create secret generic app-login-secret \
        --from-literal=username=admin \
        --from-literal=password='MyPass123!'
```

Observed output:

```
None
      secret/app-login-secret created
```

Explanation:

- `create secret generic`: Creates a generic Kubernetes Secret.
- `app-login-secret`: Secret name.
- `--from-literal`: Adds a key-value pair directly.
- `username`: Secret key.
- `password`: Secret key.

4. Verify Secret

Execute:

```
kubectl get secret app-login-secret
```

Observed output:

```
NAME               TYPE     DATA   AGE
app-login-secret   Opaque   2      15s
```

Describe the Secret:

```
kubectl describe secret app-login-secret
```

Observed output:

```
Name:         app-login-secret
Namespace:    default

Type:  Opaque

Data
====
password:  10 bytes
username:  5 bytes
```

Explanation:

- `Opaque` means a generic Secret.
- `DATA: 2` means two keys are stored.
- `describe` displays key names and byte lengths, not the actual values.

5. Create Pod YAML file

Create:

```
nano secret-env-pod.yaml
```

Add:

```
apiVersion: v1
kind: Pod
metadata:
  name: secret-env-pod
  labels:
    app: secret-demo
    tier: backend
spec:
  containers:
    - name: secret-container
      image: busybox:1.36
      command: ["sh", "-c", "sleep 3600"]
      env:
        - name: APP_USERNAME
          valueFrom:
            secretKeyRef:
              name: app-login-secret
              key: username
        - name: APP_PASSWORD
          valueFrom:
            secretKeyRef:
```

```
      name: app-login-secret
      key: password
```

6. Understand the YAML configuration

Field	Explanation
`kind: Pod`	Creates a Pod
`containers`	Defines the container
`image`	Uses BusyBox image
`command`	Keeps the container running
`env`	Defines environment variables
`valueFrom`	Gets the value from another Kubernetes resource
`secretKeyRef`	References a particular Secret key
`name: app-login-secret`	Secret name
`key: username`	Reads username value

`key: password`	Reads password value

7. Apply Pod YAML

Execute:

```
None
kubectl apply -f secret-env-pod.yaml
```

Observed output:

```
None
pod/secret-env-pod created
```

8. Verify Pod status

Execute:

```
None
kubectl get pod secret-env-pod -o wide
```

Observed output:

```
None
NAME             READY   STATUS    RESTARTS
secret-env-pod   1/1     Running   0
```

Pod successfully started on `k8s-worker-1`.

9. Verify Secret values inside the Pod

Verify username:

```
None
kubectl exec secret-env-pod -- printenv APP_USERNAME
```

Output:

```
None
admin
```

Verify password:

```
None
kubectl exec secret-env-pod -- printenv APP_PASSWORD
```

Output:

```
None
MyPass123!
```

Result: Both Secret values were successfully injected into the container as environment variables.

10. How does it work?

Secret

`app-login-secret`

secretKeyRef

Pod

`secret-env-pod`

Container environment variables

11. Real-time DevOps scenario

Suppose an application connects to a database and needs:

- Database username
- Database password

Instead of hardcoding credentials into application code or a container image, a DevOps engineer stores them in a Kubernetes Secret and injects them into the application Pod.

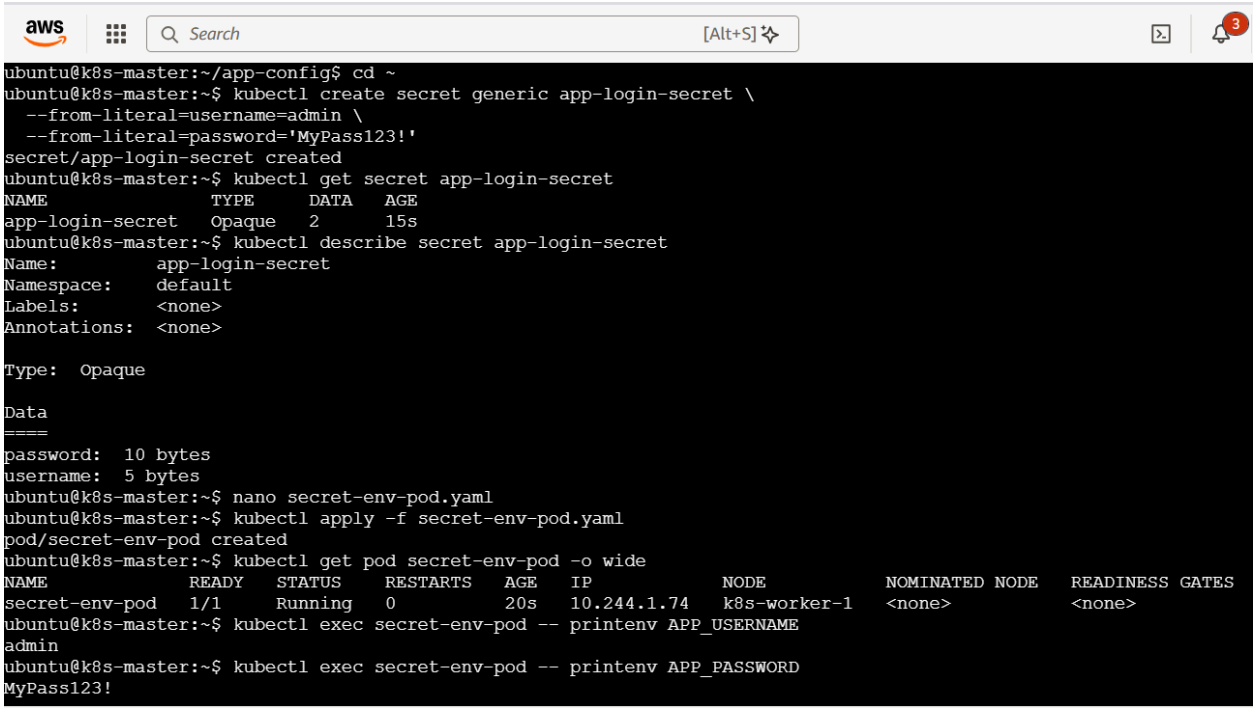
This helps separate application configuration from sensitive credentials.

12. ConfigMap vs Secret

ConfigMap	Secret
Stores non-sensitive configuration	Stores sensitive information
Example: application mode	Example: database password
`configMapRef`	`secretKeyRef`
Not intended for confidential data	Requires additional security controls

13. Final verification checklist

- Created Kubernetes Secret.
- Stored username and password.
- Verified Secret using `kubectl get`.
- Inspected Secret using `kubectl describe`.
- Created Pod YAML.
- Configured `secretKeyRef`.
- Applied Pod successfully.
- Verified Pod is Running.
- Verified username inside container.
- Verified password inside container.




```
ubuntu@k8s-master:~$ kubectl exec secret-env-pod -- printenv APP_PASSWORD
MyPass123!
ubuntu@k8s-master:~$ kubectl exec secret-env-pod -- printenv APP_USERNAME
kubectl exec secret-env-pod -- printenv APP_PASSWORD
admin
MyPass123!
ubuntu@k8s-master:~$ kubectl create secret generic app-login-secret \
  --from-literal=username=admin \
  --from-literal=password='MyPass123!'
error: failed to create secret secrets "app-login-secret" already exists
ubuntu@k8s-master:~$ kubectl get secret app-login-secret
NAME                TYPE     DATA   AGE
app-login-secret    Opaque   2      11m
ubuntu@k8s-master:~$ kubectl describe secret app-login-secret
Name:         app-login-secret
Namespace:    default
Labels:       <none>
Annotations:  <none>

Type:  Opaque

Data
====
password:  10 bytes
username:  5 bytes
```

K8S-06 Task 3 successfully completed.

4.Create a Secret using a YAML file, mount it as a volume in a pod, and verify the specific Secret values are available as files.

1. Objective

To create a Kubernetes Secret using a YAML file, mount the Secret inside a Pod as a volume, and verify that the username and password are accessible as files inside the container.

2. Environment

- Platform: AWS EC2
- Operating System: Ubuntu 24.04.5
- Kubernetes Version: v1.37.2
- Cluster: 1 Master and 2 Worker Nodes
- Container Image: `busybox:1.36`

3. Create Secret YAML file

Navigate to the home directory:

```
None
cd ~
```

Create a YAML file:

```
None
nano app-secret.yaml
```

Add the following configuration:

```
None
apiVersion: v1
kind: Secret
metadata:
  name: app-file-secret
type: Opaque
stringData:
```

```
  username: admin
  password: MyPass123!
```

Save and exit:

- Press `Ctrl + O`
- Press Enter
- Press `Ctrl + X`

Explanation:

Field	Purpose
`apiVersion: v1`	Kubernetes API version
`kind: Secret`	Creates a Kubernetes Secret
`metadata.name`	Secret name
`type: Opaque`	Generic Secret type
`stringData`	Allows plain-text input in the YAML file
`username`	Username key

`password`	Password key

`stringData` is convenient because we don't need to manually Base64-encode the values. Kubernetes converts them into Secret data when storing the object.

4. Apply the Secret YAML

Execute:

```
None
kubectl apply -f app-secret.yaml
```

Observed output:

```
None
secret/app-file-secret created
```

5. Verify the Secret

Execute:

```
None
kubectl get secret app-file-secret
```

Observed output:

```
None
NAME              TYPE     DATA   AGE
app-file-secret   Opaque   2      43s
```

Describe the Secret:

```
None
kubectl describe secret app-file-secret
```

Observed output:

```
None
      Name:         app-file-secret
      Namespace:    default

      Type:  Opaque

      Data
      ====
      password:  10 bytes
      username:  5 bytes
```

Explanation:

- `Opaque` represents a generic Secret.
- `DATA: 2` means two keys are stored.
- `username: 5 bytes` indicates the length of the username value.
- `password: 10 bytes` indicates the length of the password value.

6. Create Pod YAML file

Create a new file:

```
None
      nano secret-volume-pod.yaml
```

Add the following YAML configuration:

```
None
      apiVersion: v1
      kind: Pod
      metadata:
        name: secret-volume-pod
```

```
    labels:
      app: secret-volume-demo
      tier: backend
  spec:
    containers:
      - name: secret-container
        image: busybox:1.36
        command: ["sh", "-c", "sleep 3600"]
        volumeMounts:
          - name: secret-volume
            mountPath: /etc/app-secrets
            readOnly: true
    volumes:
      - name: secret-volume
        secret:
          secretName: app-file-secret
```

Save and exit the editor.

7. Understand the Pod YAML

YAML field	Explanation
`kind: Pod`	Creates a Pod
`containers`	Defines the container
`image`	Uses BusyBox image

`command`	Keeps the container running
`volumeMounts`	Mounts a volume inside the container
`name: secret-volume`	Volume mount name
`mountPath`	Directory where Secret files are available
`readOnly: true`	Prevents writing through this mount
`volumes`	Defines the Pod volume
`secret`	Uses a Secret as the volume source
`secretName`	References `app-file-secret`

8. Apply the Pod YAML

Execute:

```
None
kubectl apply -f secret-volume-pod.yaml
```

Observed output:

```
None
pod/secret-volume-pod created
```

9. Verify Pod status

Execute:

```
None
kubectl get pod secret-volume-pod -o wide
```

Observed output:

```
None
NAME                READY   STATUS    RESTARTS
secret-volume-pod   1/1     Running   0
```

Pod was successfully scheduled on `k8s-worker-1`.

10. Verify Secret files inside the container

First, list the mounted files:

```
None
kubectl exec secret-volume-pod -- ls -l /etc/app-secrets
```

Observed output:

```
None
total 0
password -> ..data/password
username -> ..data/username
```

Explanation:

Kubernetes creates symbolic links for mounted Secret files. This is normal behavior.

Now verify the username:

```
None
      kubectl exec secret-volume-pod -- cat
      /etc/app-secrets/username
```

Output:

```
None
      admin
```

Verify the password:

```
None
      kubectl exec secret-volume-pod -- cat
      /etc/app-secrets/password
```

Output:

```
None
      MyPass123!
```

Both Secret files verified

11. How does Secret volume mounting work?

Secret YAML file

`app-secret.yaml`

Kubernetes Secret

`app-file-secret`

Pod volume

`volumes` + `volumeMounts`

Container files

`/etc/app-secrets/username`

`/etc/app-secrets/password`

12. Task 3 vs Task 4

Feature	Task 3	Task 4
Secret creation	CLI command	YAML file
Secret injection	Environment variables	Volume mount
YAML field	`secretKeyRef`	`secret` and `volumeMounts`
Access command	`printenv`	`cat`
Result	Values available as environment variables	Values available as files

13. Real-time DevOps scenario

Imagine an application needs database credentials.

A DevOps engineer can store the username and password in a Kubernetes Secret and mount them as files inside the application container.

The application can read these files when it needs to authenticate with the database.

This approach avoids hardcoding credentials directly into the application image.

Security note: Kubernetes Secret values are Base64-encoded by default, not automatically encrypted. Protect YAML files containing credentials, enable encryption at rest, and use RBAC to restrict access.

14. Final verification checklist

- Created Secret YAML file.
- Used `stringData` for username and password.
- Applied Secret successfully.
- Verified Secret using `kubectl get`.
- Inspected Secret using `kubectl describe`.
- Created Pod YAML file.
- Configured Secret volume.
- Mounted Secret at `/etc/app-secrets`.
- Applied Pod successfully.
- Verified Pod is Running.
- Verified username file.
- Verified password file.

```
ubuntu@k8s-master:~$ cd ~
ubuntu@k8s-master:~$ nano app-secret.yaml
ubuntu@k8s-master:~$ cat app-secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: app-file-secret
type: Opaque
stringData:
  username: admin
  password: MyPass123!
ubuntu@k8s-master:~$ kubectl apply -f app-secret.yaml
secret/app-file-secret created
ubuntu@k8s-master:~$ kubectl get secret app-file-secret
NAME              TYPE     DATA   AGE
app-file-secret   Opaque   2      43s
ubuntu@k8s-master:~$ kubectl describe secret app-file-secret
Name:         app-file-secret
Namespace:    default
Labels:       <none>
Annotations:  <none>

Type:  Opaque

Data
====
password:  10 bytes
username:  5 bytes
ubuntu@k8s-master:~$ nano secret-volume-pod.yaml
ubuntu@k8s-master:~$ kubectl apply -f secret-volume-pod.yaml
```

```
ubuntu@k8s-master:~$ kubectl describe secret app-file-secret
Name:         app-file-secret
Namespace:    default
Labels:       <none>
Annotations:  <none>

Type:  Opaque

Data
====
password:  10 bytes
username:  5 bytes
ubuntu@k8s-master:~$ nano secret-volume-pod.yaml
ubuntu@k8s-master:~$ kubectl apply -f secret-volume-pod.yaml
pod/secret-volume-pod created
ubuntu@k8s-master:~$ kubectl get pod secret-volume-pod -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP            NODE          NOMINATED NODE   READINESS GATES
secret-volume-pod   1/1     Running   0          16s   10.244.1.75   k8s-worker-1   <none>           <none>
ubuntu@k8s-master:~$ kubectl exec secret-volume-pod -- ls -l /etc/app-secrets
total 0
lrwxrwxrwx    1 root     root            15 Nov 12 11:32 password -> ..data/password
lrwxrwxrwx    1 root     root            15 Nov 12 11:32 username -> ..data/username
ubuntu@k8s-master:~$ kubectl exec secret-volume-pod -- cat /etc/app-secrets/username
adminubuntu@k8s-mastekubectl exec secret-volume-pod -- cat /etc/app-secrets/passwordsword
MyPass123!ubuntu@k8s-master:~$
```

K8S-06 Task 4 successfully completed

5.Inject a ConfigMap as environment variables and a Secret as files into the same pod, ensuring both are accessible within the pod.

1. Objective

To use ConfigMap and Secret together in a single Kubernetes Pod.

- ConfigMap values are injected as environment variables.
- Secret values are mounted as files inside the container.

2. Existing resources used

Resource	Name	Purpose
ConfigMap	`app-env-config`	Stores application configuration
Secret	`app-file-secret`	Stores username and password

Pod	`config-secret-combined-pod`	Uses both resources

3. Pod YAML configuration

File: `config-secret-combined-pod.yaml`

```
None
apiVersion: v1
kind: Pod
metadata:
  name: config-secret-combined-pod
  labels:
    app: config-secret-demo
    tier: backend
spec:
  containers:
    - name: application-container
      image: busybox:1.36
      command: ["sh", "-c", "sleep 3600"]
      envFrom:
        - configMapRef:
            name: app-env-config
      volumeMounts:
        - name: secret-volume
          mountPath: /etc/app-secrets
          readOnly: true
  volumes:
    - name: secret-volume
      secret:
        secretName: app-file-secret
```

4. Apply the configuration

```
None
kubectl apply -f config-secret-combined-pod.yaml
```

Output:

```
None
pod/config-secret-combined-pod created
```

5. Verify Pod status

```
None
kubectl get pod config-secret-combined-pod -o wide
```

Observed result:

Property	Value
Pod name	config-secret-combined-pod
Ready	1/1
Status	Running
Restarts	0
Pod IP	10.244.1.76
Worker node	k8s-worker-1

6. Verify ConfigMap environment variables

Commands:

```
None
kubectl exec config-secret-combined-pod -- printenv APP_MODE
kubectl exec config-secret-combined-pod -- printenv APP_COLOR
kubectl exec config-secret-combined-pod -- printenv DB_HOST
kubectl exec config-secret-combined-pod -- printenv DB_PORT
```

Observed output:

```
None
production
blue
mysql-service
3306
```

This confirms that Kubernetes injected ConfigMap values into the container's environment.

7. Verify Secret volume mounting

List the mounted files:

```
None
kubectl exec config-secret-combined-pod -- ls -l /etc/app-secrets
```

Observed output:

```
None
password -> ..data/password
username -> ..data/username
```

These symbolic links are normal Kubernetes Secret volume behaviour.

Read the username:

```
None
kubectl exec config-secret-combined-pod -- cat
/etc/app-secrets/username
```

Output:

```
None
admin
```

Read the password:

```
None
kubectl exec config-secret-combined-pod -- cat
/etc/app-secrets/password
```

Output:

```
None
MyPass123!
```

This confirms that Kubernetes mounted Secret values as files inside the container.

8. Understand the complete workflow

ConfigMap: app-env-config

APP_MODE, APP_COLOR, DB_HOST, DB_PORT

Environment variables using envFrom

Application Pod

`config-secret-combined-pod`

Secret volume mounted at /etc/app-secrets

Secret: app-file-secret

username, password

9. Important concepts learned

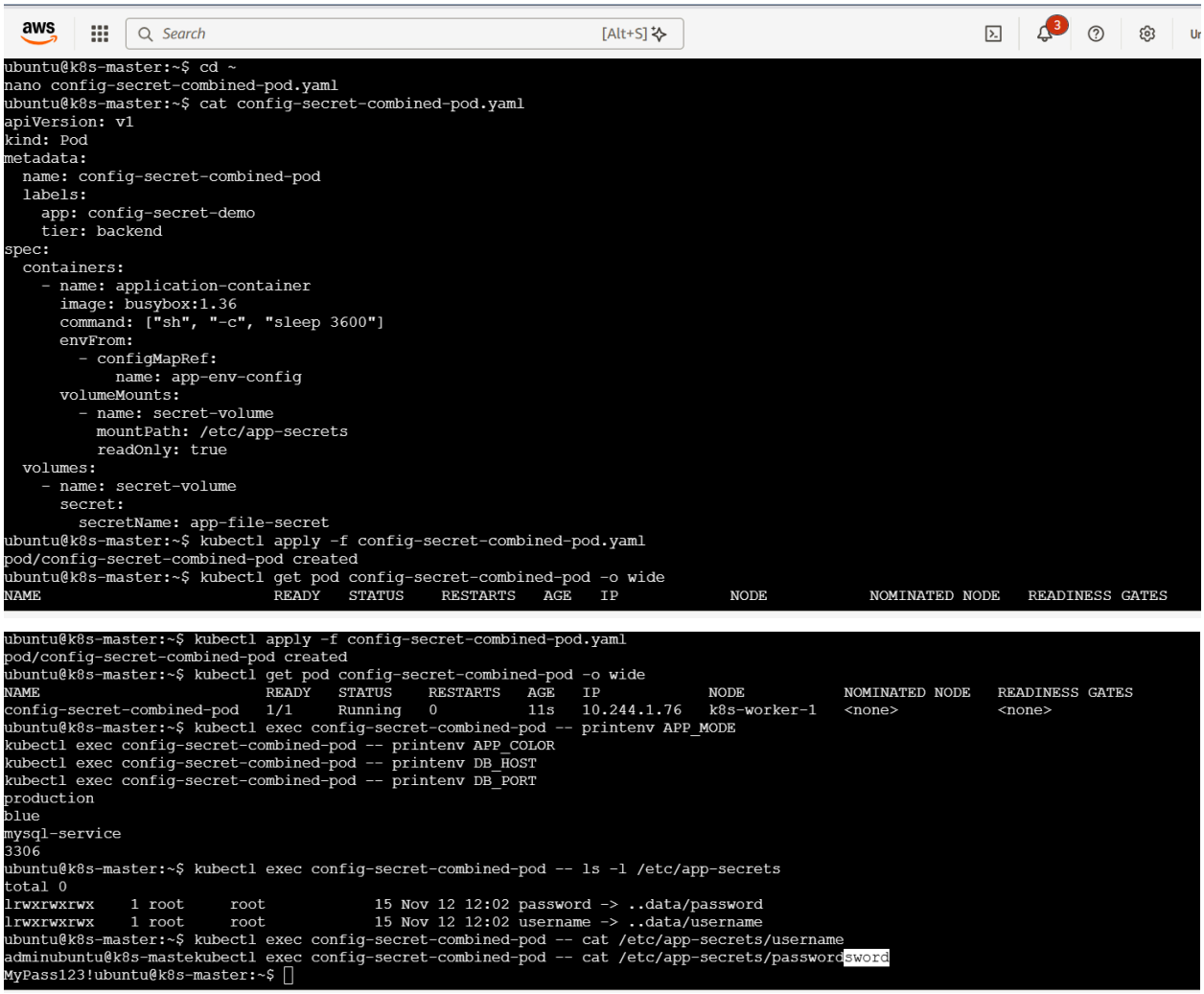
Concept	Explanation
ConfigMap	Stores non-sensitive application configuration
Secret	Stores sensitive configuration such as passwords
`envFrom`	Injects all ConfigMap keys as environment variables
`volumeMounts`	Mounts files inside a container
`mountPath`	Location where files appear inside the container
`readOnly: true`	Prevents the container from modifying the mounted files
`secretName`	Identifies which Secret to mount

Real-time DevOps example:

Imagine deploying a web application that connects to a MySQL database.

- ConfigMap provides database hostname, port and application mode.
- Secret provides database username and password.
- Kubernetes makes both available to the application container without hardcoding them in the application image.

Security reminder: Kubernetes Secrets are not automatically encrypted just because they are Secrets. Base64 encoding is not encryption. Production clusters should use encryption at rest, appropriate RBAC restrictions and secure secret-management practices.


aws
Search
[Alt+S]
3
ubuntu@k8s-master:~$ cd ~
nano config-secret-combined-pod.yaml
ubuntu@k8s-master:~$ cat config-secret-combined-pod.yaml
apiVersion: v1
kind: Pod
metadata:
name: config-secret-combined-pod
labels:
app: config-secret-demo
tier: backend
spec:
containers:
- name: application-container
image: busybox:1.36
command: ["sh", "-c", "sleep 3600"]
envFrom:
- configMapRef:
name: app-env-config
volumeMounts:
- name: secret-volume
mountPath: /etc/app-secrets
readOnly: true
volumes:
- name: secret-volume
secret:
secretName: app-file-secret
ubuntu@k8s-master:~$ kubectl apply -f config-secret-combined-pod.yaml
pod/config-secret-combined-pod created
ubuntu@k8s-master:~$ kubectl get pod config-secret-combined-pod -o wide
NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES
ubuntu@k8s-master:~$ kubectl apply -f config-secret-combined-pod.yaml
pod/config-secret-combined-pod created
ubuntu@k8s-master:~$ kubectl get pod config-secret-combined-pod -o wide
NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES
config-secret-combined-pod 1/1 Running 0 11s 10.244.1.76 k8s-worker-1 <none> <none>
ubuntu@k8s-master:~$ kubectl exec config-secret-combined-pod -- printenv APP_MODE
kubectl exec config-secret-combined-pod -- printenv APP_COLOR
kubectl exec config-secret-combined-pod -- printenv DB_HOST
kubectl exec config-secret-combined-pod -- printenv DB_PORT
production
blue
mysql-service
3306
ubuntu@k8s-master:~$ kubectl exec config-secret-combined-pod -- ls -l /etc/app-secrets
total 0
lrwxrwxrwx 1 root root 15 Nov 12 12:02 password -> ..data/password
lrwxrwxrwx 1 root root 15 Nov 12 12:02 username -> ..data/username
ubuntu@k8s-master:~$ kubectl exec config-secret-combined-pod -- cat /etc/app-secrets/username
adminubuntu@k8s-mastekubectl exec config-secret-combined-pod -- cat /etc/app-secrets/passwordsword
MyPass123!ubuntu@k8s-master:~$